

Unary Operator Overloading

L26 – L27 • CLO 2, 3 • Week 14

Topics: operator Keyword • Overloading Unary Operators

Operator Arguments • Operator Return Values

Riphah International College – ADP Computing

Prepared for Students | 2024–25

■ Topic	■ What You Will Learn
L26 – operator Keyword	What operator overloading is and syntax of the operator keyword
L26 – Overloading Unary Ops	How to overload ++, --, -, ! and similar unary operators
L27 – Operator Arguments	How many arguments unary operators take (member vs. friend)
L27 – Operator Return Values	What to return from an overloaded operator (value vs. reference)

1. Operator Overloading — Concept & Definition

What is Operator Overloading?

Operator Overloading is a feature of C++ that allows programmers to **redefine the meaning of built-in operators** (like +, -, ++, --) for user-defined types (classes/structs). Instead of calling a function explicitly, you can use natural operator syntax on objects.

■ Formal Definition

Operator overloading is the ability to give additional meaning to an existing C++ operator when it is applied to user-defined data types, without changing its meaning for built-in types. This is achieved by defining a special function using the **operator** keyword.

The operator Keyword — Syntax

To overload an operator in C++, you define a function whose name begins with the reserved keyword **operator** followed directly by the operator symbol.

```
// General Syntax
returnType operator Symbol ( parameters )

// Example: overload unary minus (-) for class Counter
```

```
Counter operator- ( );
// 'operator' = keyword | '-' = symbol | '()' = no parameter (unary member)
```

Why Do We Need It?

Without overloading:

- obj3 = obj1.add(obj2) ← ugly, verbose
- obj1.increment() ← not intuitive
- Must remember method names for each class

With overloading:

- obj3 = obj1 + obj2 ← clean, readable
- ++obj1 ← natural syntax
- Code looks like math — easy to understand

Real-Life Analogy

Temperature App Example

Imagine a Temperature class representing degrees Celsius. If you want to negate a temperature (e.g., convert +30°C to -30°C), it is much more natural to write **-temp** than **temp.negate()**. Operator overloading lets you use the familiar minus sign directly on your object.

Operators That CAN Be Overloaded vs Cannot

Can Overload	Cannot Overload
+ - * / % ^ & ~ ! = < > += -=	:: (scope resolution)
*= /= %= ^= &= = << >> <=> >>= == !=	. (member access)
<= >= && ++ -- , ->* -> () []	.* (pointer-to-member)
new delete new[] delete[]	?: (ternary / conditional)

2. What is a Unary Operator?

■ Definition — Unary Operator

A **unary operator** is an operator that operates on **only one operand** (one value/variable). It performs an operation on a single object or value, unlike binary operators which need two operands.

■ Definition — Unary Operator

A **unary operator** is an operator that operates on **only one operand** (one value/variable). It performs an operation on a single object or value, unlike binary operators which need two operands.

Common Unary Operators in C++

Operator	Name	Built-in Usage	What it does
++	Increment	++x or x++	Adds 1 to the value
--	Decrement	--x or x--	Subtracts 1 from the value
-	Unary Minus	-x	Negates the value
!	Logical NOT	!flag	Flips true/false
~	Bitwise NOT	~bits	Flips all bits

3. Overloading Unary Operators — Step by Step

How to Overload a Unary Operator (Member Function)

How to Overload a Unary Operator (Member Function)

When overloaded as a **member function**, a unary operator takes **zero explicit parameters** — the object it acts on is passed implicitly via the **this** pointer.

```
// ■■■ BASIC EXAMPLE: Overloading unary minus (-) ■■■■
```

```
#include <iostream>

using namespace std;

class Distance {
    int meters;

public:
    Distance(int m) : meters(m) {} // constructor

// ■■■ Overload unary minus ■■■■

    Distance operator-() { // 0 parameters (member)
        return Distance(-meters); // negate & return new obj
    }

    void display() { cout << meters << " m" << endl; }
};

int main() {
    Distance d1(50); // d1 = +50 m
    Distance d2 = -d1; // calls operator-()

    d1.display(); // Output: 50 m
    d2.display(); // Output: -50 m

    return 0;
}
```

■ How It Works — Step by Step

1. **-d1** is written in `main()`
2. Compiler sees: operand is a Distance object, operator is unary -
3. Compiler calls **d1.operator-()** automatically
4. Inside the function, **meters** refers to d1's meters (via this pointer)
5. A new Distance(-50) is created and returned
6. That new object is assigned to d2

Overloading Prefix ++ (Pre-increment)

[illegible]

Overloading Postfix ++ (Post-increment)

To distinguish postfix **obj++** from prefix **++obj**, C++ uses a **dummy int parameter** in the postfix version. You never actually pass a value — it is just a signal to the compiler.

```
// ■■■ Postfix increment: obj++ ■■■■
Counter operator++(int) { // POSTFIX: dummy int parameter
Counter temp = *this; // save current state
++count; // increment
return temp; // return OLD value (before increment)
}

int main() {
Counter c(5);
c++; // calls c.operator++(0) – dummy 0 passed by compiler
c.show(); // Output: 6 (but expression c++ evaluates to 5)
}
```

Prefix vs Postfix — Quick Comparison

Feature	Prefix ++obj	Postfix obj++
Signature	operator++()	operator++(int)
Extra param	None	Dummy int (not used)
Returns	Updated object (*this)	Copy of old object

Efficiency	Faster (no temp copy)	Slower (needs temp)
Expression val	New (incremented) value	Old (original) value

4. Operator Arguments (L27)

■ Definition — Operator Arguments

The **arguments (parameters)** of an overloaded operator function describe what objects/values the operator receives as input. For **unary operators**, the number of arguments depends on whether the operator is defined as a **member function** or a **friend (non-member) function**.

■ Definition — Operator Arguments

The **arguments (parameters)** of an overloaded operator function describe what objects/values the operator receives as input. For **unary operators**, the number of arguments depends on whether the operator is defined as a **member function** or a **friend (non-member) function**.

Rule: Arguments for Unary Operators

Implementation	Parameters Needed	Reason
Member Function	0 (zero explicit params)	Object available via this pointer
Friend / Non-member	1 (one explicit param)	No this — object must be passed in
Postfix (member)	1 dummy int	Distinguishes postfix from prefix
Postfix (friend)	2: object + dummy int	Object + dummy to signal postfix

Member vs Friend — Code Comparison

```
// ■■■ Member Function Version (0 parameters) ■■■■  
  
class Counter {  
    int count;  
public:  
    Counter(int c=0): count(c) {}  
    Counter operator-() { // 0 params – uses 'this' internally  
        return Counter(-count);  
    }  
};  
  
// ■■■ Friend Function Version (1 explicit parameter) ■■■■  
  
class Counter2 {  
    int count;  
public:  
    Counter2(int c=0): count(c) {}  
    friend Counter2 operator-(Counter2 obj); // 1 param – no 'this'  
};  
  
Counter2 operator-(Counter2 obj) { // defined outside class  
    return Counter2(-obj.count);  
}
```

5. Operator Return Values (L27)

■ Definition — Operator Return Values

The **return value** of an overloaded operator is what the expression evaluates to after the operator is applied. Choosing the correct return type is critical — it must match how the operator behaves on built-in types and how the result will be used in expressions.

■ Definition — Operator Return Values

The **return value** of an overloaded operator is what the expression evaluates to after the operator is applied. Choosing the correct return type is critical — it must match how the operator behaves on built-in types and how the result will be used in expressions.

Return Type Guidelines for Unary Operators

[illegible]

7. Real-Life Benefits — Where It Helps Us

■ Scientific / Mathematical Libraries

Math libraries (like Eigen for matrices or complex number types) use unary `-` to negate vectors/matrices. Without it, negating a 3D vector would require calling a verbose function like `v.negate()` instead of simply `-v`.

■ Game Development

A `Vector3D` class representing a player's movement direction overloads unary `-` to reverse direction instantly: `-velocity` gives opposite movement. This makes game physics code natural and readable.

■ Financial Software

A `Money` or `Currency` class overloads unary `-` to represent negative amounts (debts/expenses). Prefix `++` and `--` adjust amounts by a unit step. This is used in accounting systems and trading platforms.

■ Date & Time Classes

A `Time` class can overload prefix `++` to advance time by one second and `--` to go back. This makes loops like `while (++timer != end)` clean and expressive in scheduling or simulation software.

■ Physics Simulations

Temperature, voltage, or force objects overload unary `-` to negate direction. This avoids writing separate negation methods and keeps simulation code consistent with mathematical notation.

8. Summary Cheat-Sheet

Concept	Key Rule / Syntax	Remember
operator keyword	<code>returnType operator symbol (params)</code>	Must use keyword 'operator' before symbol
Unary operator	Works on 1 operand only	<code>++ -- - ! ~</code> are all unary
Member unary	0 parameters (uses <code>this</code>)	Object calling it = the operand
Friend unary	1 parameter (object passed in)	No <code>this</code> pointer available
Prefix ++	<code>operator++()</code> → <code>return *this</code> (ref)	Increment first, then return
Postfix ++	<code>operator++(int)</code> → <code>return old (val)</code>	Save old, increment, return old
Return value	Prefix → ref; Others → new object	Prefix faster; Postfix needs copy
Cannot overload	<code>:: . .* ?: sizeof</code>	5 operators are permanently fixed

9. Common Mistakes to Avoid

■ Wrong #1: Too many parameters for member unary

Mistake: `operator--(int x)` ← named param not needed

Fix: `operator--(int)` ← dummy, unnamed, not used

■ Wrong #2: Postfix returning reference

Mistake: `Counter& operator++(int)` ← returns ref to local = dangling ref!

Fix: `Counter operator++(int)` ← return by value (copy of old state)

■ Wrong #3: Prefix NOT returning reference

Mistake: `Counter operator++()` ← returns a copy, chaining like `++++x` breaks

Fix: `Counter& operator++()` ← return `*this` by reference

■ Wrong #4: Overloading operators that cannot be overloaded

Mistake: trying to overload `?:` or `::` or `.`

Fix: Check the allowed list — those 5 are permanently reserved